UNIVERSITY OF PLYMOUTH
School of Engineering, Computing & Mathematics

Session 3: Connecting Containers - JS Objects

Full-Stack Development

Mark Dixon

School of Engineering, Computing and Mathematics

Scrum Standup: Last Session

- What did we do last session

  - Progress:
    did you learn anything (was anything particularly useful or good)?
    (write down topics, what you can remember)

  - Problems / Blockers:

    How did the lab session go (were you able to get something running)?

    What could be better?

Introduction

Today's topics

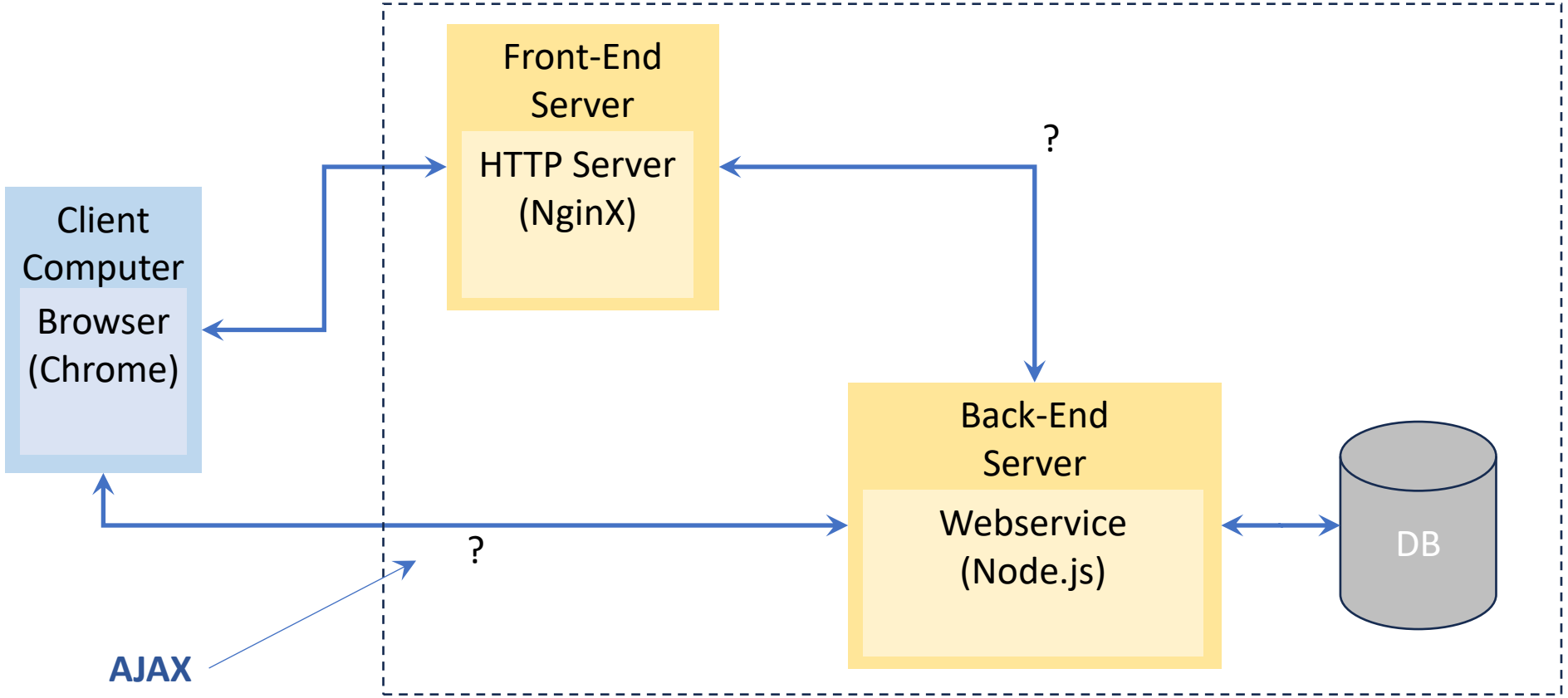
1. Connecting Containers / Components
    1. Ajax & CORS
    2. JavaScript OOP & JSON

Session learning outcomes – **by the end of today's lecture you will be able to:**

- Use Ajax to connect containers:
    - client-side JS code (running in browser) directly to the backend
    - server-side JS code to another server / container
- Use bespoke objects to represent data within your own JS code
- Use JSON to send objects between containers

Ajax – Communication between containers

- Sample component architecture (this can vary considerably):



Ajax

Asynchronous JavaScript And XML

Misleading name (now)

- No need for XML
- JSON more common
- No need for asynchrony

Browsers support creating XMLHttpRequest object

- Allow JS to make a server query without loading a new page
- Can get access to HTTP response in code

Allows:

- Update web page without reloading the page
- Request data from server - after page has loaded
- Receive data from server - after page has loaded
- Send data to a server - in the background

Ajax – `XMLHttpRequest` object

Sending an Ajax request

- Method – GET/POST
- URL – server file path
- async – true/false

Methods

- `open` – prepares the request for sending
- `send` – send the request
- …

Receiving a response

- Ready state
  - 0 – request not initialized
  - 1 – server connection established
  - 2 – request received
  - 3 – processing request
  - 4 – request finished and response ready
- HTTP Response Status codes
  - 0 – timeout
  - 200 – OK
  - 404 – page not found
  - 500 – internal server error

Ajax

main.js

```
window.onload = function() {
  let request = new XMLHttpRequest();

  request.onreadystatechange = function() {
    if (this.readyState === 4 && this.status == 200) {
      let elem = document.getElementById("spnNum");
      elem.innerHTML = this.responseText;
    }
  };

  request.open("GET", "http://localhost:82/", true);
  request.send();
}
```

1. Initialise new **XMLHttpRequest** object
2. Create event handler callback function
3. Send the request

Ajax – Docker Compose for demo project

```
DEMOAJAX
  app-backend
    run.sh
    server.js
  app-frontend
    index.html
    main.css
    main.js
  compose.yaml
  up.sh
```

compose.yaml > { } services

```yaml
name: demo-ajax
services:

  frontend:
    image: nginx
    volumes:
      - ./app-frontend:/usr/share/nginx/html:ro
    ports:
      - "81:80"

  backend:
    image: node:22.9.0
    volumes:
      - ./app-backend:/usr/src/app
    working_dir: /usr/src/app
    ports:
      - "82:9000"
    command: sh -c "
      chmod +x run.sh &&
      ./run.sh"
```

Front-End Server

HTTP Server (NginX)

Back-End Server

Webservice (Node.js)

Ajax

WARNING: does not work yet

Front-end (client-side)

index.html

```html
<html>
  <head>
    <title>Random Number</title>
    <script src="main.js"></script>
  </head>
  <body>
    Number: <span id="spnNum">loading...</span>
  </body>
</html>
```

main.js

```javascript
window.onload = function() {
  let request = new XMLHttpRequest();
  request.onreadystatechange = function() {
    if (this.readyState === 4 && this.status == 200) {
      let elem = document.getElementById("spnNum");
      elem.innerHTML = this.responseText;
    }
  };
  request.open("GET", "http://localhost:82/", true);
  request.send();
}
```

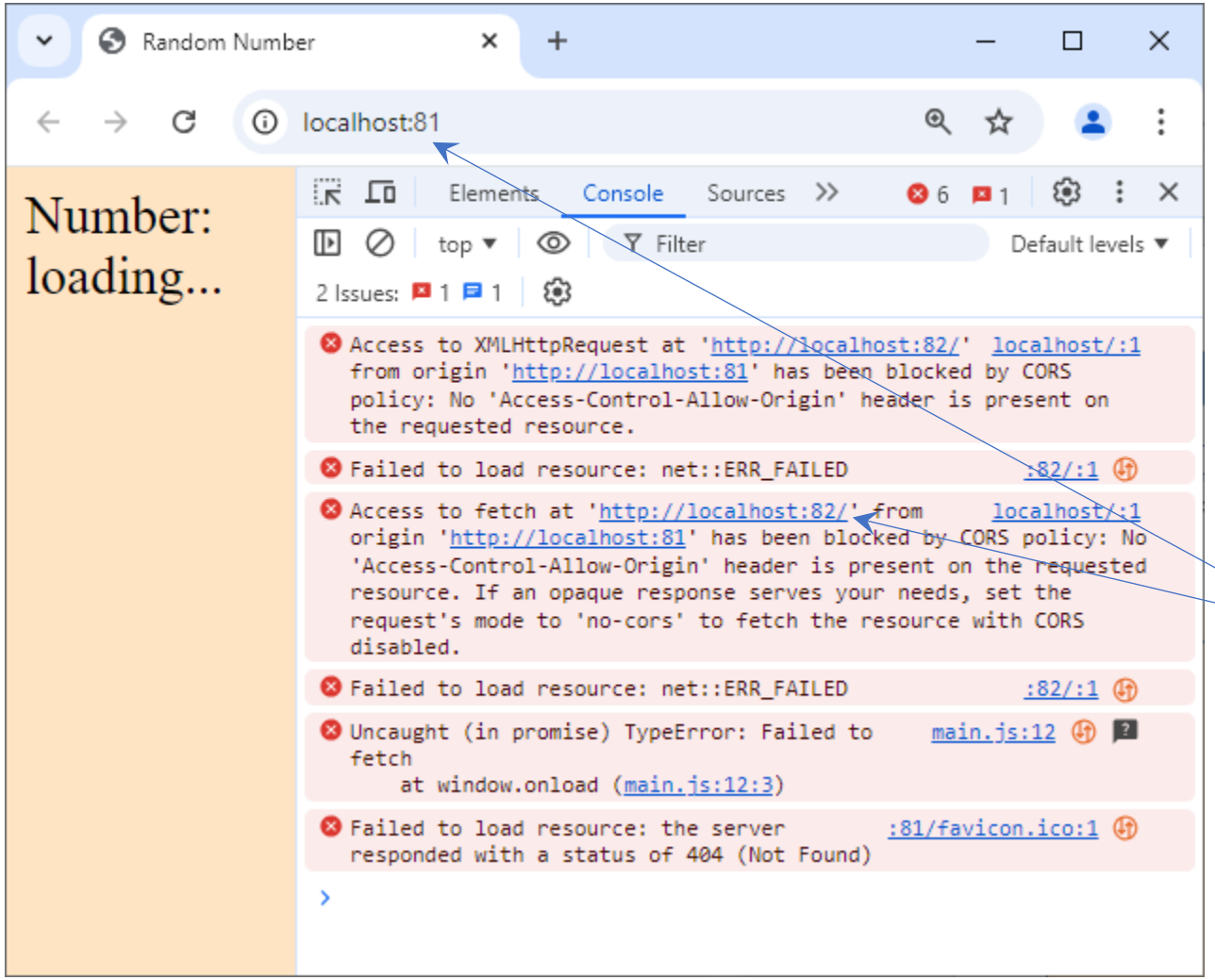
- Send request to another server (e.g. for random number)
- Inject response (actual random number) into current web-page

Back-end (server-side)

server.js

```javascript
let http = require("http");
let port = 9000;

let server = http.createServer(
  function(request, response) {
    var num = Math.random();
    response.end(num.toString());
  }
);

server.listen(port, function() {
  console.log("Server listening on port " + port);
});
```

Ajax - CORS (Cross Origin Resource Sharing)



- Page loads, but request fails
- Press F12 in browser
- Click Console
- Can see that CORS is blocking

- CORS blocks requests to different
  - Domains, and
  - **Ports**

Ajax – CORS Fix

Fixed

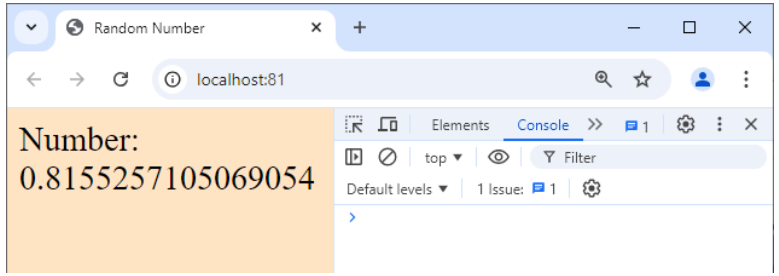
Front-end (client-side)

index.html

```html
<html>
  <head>
    <title>Random Number</title>
    <script src="main.js"></script>
  </head>
  <body>
    Number: <span id="spnNum">loading...</span>
  </body>
</html>
```

main.js

```js
window.onload = function() {
  let request = new XMLHttpRequest();
  request.onreadystatechange = function() {
    if (this.readyState === 4 && this.status == 200) {
      let elem = document.getElementById("spnNum");
      elem.innerHTML = this.responseText;
    }
  };
  request.open("GET", "http://localhost:82/", true);
  request.send();
}
```



- **Back-end (server-side)** must give permission for other domains/ports to send requests

Back-end (server-side)

server.js

```js
let http = require("http");
let port = 9000;

let server = http.createServer(
  function(request, response) {
    var num = Math.random();
    var referrer = "http://localhost:81";
    response.setHeader('Access-Control-Allow-Origin', referrer);
    response.end(num.toString());
  }
);

server.listen(port, function() {
  console.log("Server listening on port " + port);
});
```

Avoid:

```js
response.setHeader('Access-Control-Allow-Origin', "*");
```

Ajax – advantages and disadvantages

Advantages

- Allows webpage content to be updated without page reload
- Faster response times – only necessary data transfer
- Reduced Server Load (due to partial page load) - improved performance
- Based on web standards (such as JavaScript, XML…)
- Asynchronous communication between client & server

Disadvantages

- Security – vulnerable without enhancement (such as authentication & HTTPS)
- user cannot tell that content is loading
- page invisible for browser navigation & web crawlers
  - inhibits SEO (Search Engine Optimisation)
- CORS
- May be blocked by browser

Object-oriented programming

The solution to a problem is

- A collection of objects with individual attributes and responsibilities
- Objects interact through “message passing”

Advantages

- Encapsulation/information hiding
- Intuitive
- Reuse

Weaknesses

- Can be overkill for limited applications
- Can lead to spaghetti code

Objects

An unordered collection of data with associated keys

- Objects have **methods**, **inherit** from other objects and are **dynamic**

Objects attributes

- Object prototype (references the object properties are inherited from)
- class (a string categorising the object type)
- Extensible flag (specifies whether new properties can be added)

Object categories

- Native object (defined by the browser – arrays, functions, dates…)
- Host object (e.g., HTML elements)
- User-defined objects (created by executing JS code)

Properties & values

Properties

- Unique identifier for values
- Any string (including the empty string)
- Property attributes
  - Writeable
  - Enumerable
  - Configurable

Values

- Any valid JavaScript value, or a method

JavaScript objects

Objects have keys and values (associative arrays, hashmaps)

```
person = {
    name: "Barry",
    age: 42,
}
```

Dot notation for access

```
person.email = "barry@plymouth.ac.uk"
```

You can also loop through object properties

```
for (property in person) {
    if (person.hasOwnProperty(property)) {
        console.log(property + ": " + person[property]);
    }
}
```

JavaScript strings

```
let str = "Hello World";
```

- JavaScript strings are a collection of characters
- No char type – just a string with a single character
- All strings have a property called length
- Strings can be created as objects using

```
new String("Hello World");
```

but this is much slower than using primitive strings

JavaScript arrays

Declaration

```
things = ["Barry", 42, "barry@plymouth.ac.uk"]
```

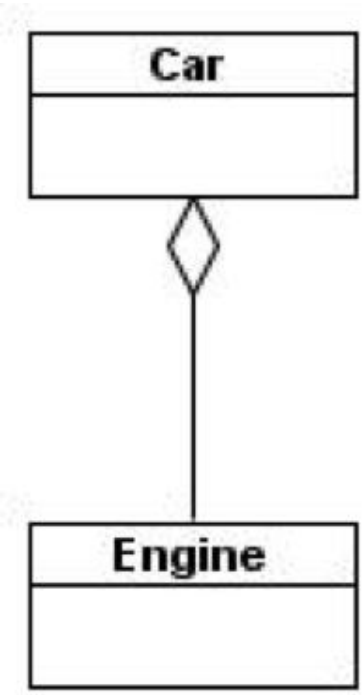
Loop

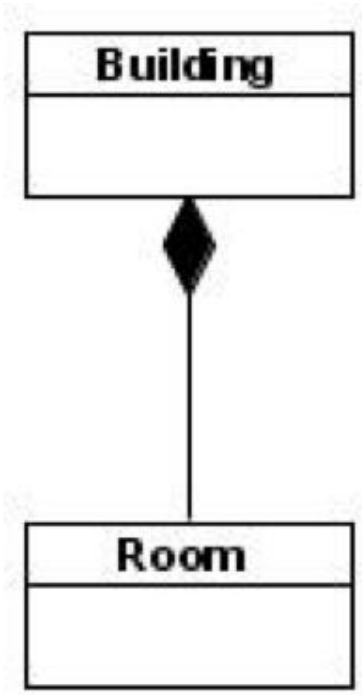
```
for (index=0; index<things.length; index++) {
    text += things[index];
}
```

Extra properties

- length, sort(), push(), indexOf("Barry");

Object Association: Aggregation & Composition






```
let myEngine = {
    type: "diesel",
    volume: 2.1,
    weight: 35
}

let car = {
    type: "fiat",
    engine: myEngine,
    colour: "green"
}

console.log(car.engine.type);
```

Engine can exist without Car (Aggregation)

Room cannot exist without Building (Composition)

Object instantiation – named functions

- JavaScript functions can be used as **constructors**
- Function instantiations work as objects

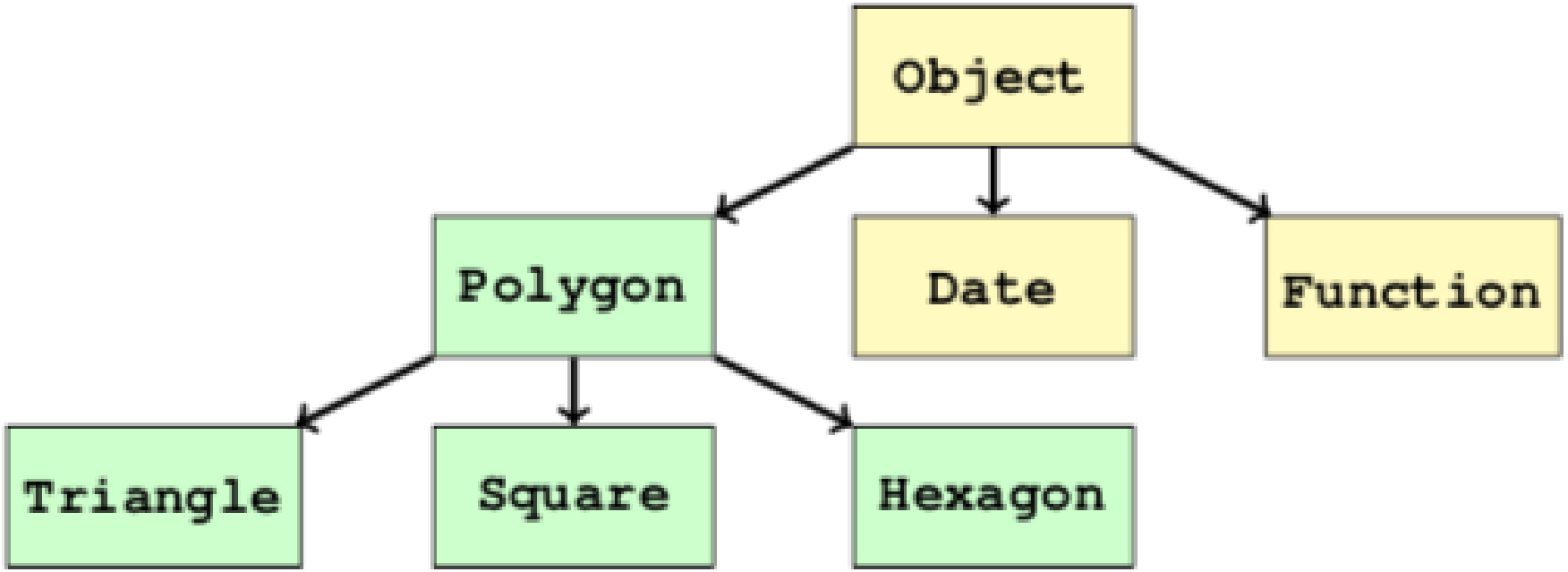
```
function World(population) {
    this.population = population;
    this.rotation = 1;

    this.rotate = function(d) {
        this.rotation = this.rotation + d;
    }
}

let earth = new World(5000000000)
earth.rotate(90)

let mars = new World(0);
```

Function prototypes

When JS object is declared it creates a related prototype data structure containing important function parameters and sub-functions

- Name (may be empty), number of arguments, call, apply
- Accessible through the instance variable “prototype”

Prototypes are shared between all instances



Inheritance: DRY (Do not Repeat Yourself) principle

Allows a class to access methods from another class

```
function Rectangle(width, height) {
    this.width = width;
    this.height = height;
}

Rectangle.prototype.computeArea = function() {
    return this.width * this.height;
}

function Square(width) {
    Rectangle.call(this, width, width);
}

Square.prototype = Object.create(Rectangle.prototype);
```

Overriding

Provides **new code** for an **existing superclass method**

```
function Rectangle(width, height) {
    this.width = width;
    this.height = height;
}

Rectangle.prototype.computeArea = function() {
    return this.width * this.height;
}

function Square(width) {
    Rectangle.call(this, width, width);
}

Square.prototype = Object.create(Rectangle.prototype);

Square.prototype.computeArea = function() {
    return Math.pow(this.width, 2);
}
```

Extending a superclass

Uses a superclass method and adds functionality

```
Rectangle.prototype.computeArea = function() {
    return this.width * this.height;
}

Square.prototype.computeArea = function() {
    console.log("Do something new here");
    return Rectangle.prototype.computeArea.call(this);
}
```

Polymorphism

- A single interface to entities of different types
- Superclass can be used for array membership
- Trivial in JavaScript with no type checking

```
window.onload = function() {
    arr = [];
    arr[0] = new Rectangle(5, 12);
    arr[1] = new Square(6);

    for (i=0; i<arr.length; i++) {
        console.log(arr[i].computeArea());
    };
```

Information hiding

- closure of the constructor function is only accessible to the functions defined within the constructor function
- Variables local to the constructor function can be used as private variables

```
function Counter(start) {
    let count = start;

    return {
        increment: function() {
            count++;
        },
        get: function() {
            return count;
        }
    }
}

let foo = Counter(4);
```

Extending the class

```
class Square extends Rectangle {
    constructor(width) {
        super(width, width);
    }
}

window.onload = function() {
    let squareInstance = new Rectangle(4);
    console.log(squareInstance.computeArea());
};
```

- Specify the class being extended with the **extends** keyword
- Reference the parent class with **super**

JSON

JavaScript **Object** Notation

```
{ "modules":
    [
        {"code": "COMP2005",
         "name": "Software Development Tools and Practices",
         "stage": 2},

        {"code": "COMP3006",
         "name": "Full-Stack Development",
         "stage": 4},
    ]
}
```

- Serialisable data that can be converted directly into a JS object in your program
- Widely used for data exchange online

JSON

- text format
- completely language independent
- uses conventions familiar to C-family of languages (including C, C++, C#, Java, JavaScript, Perl, Python …)

based on JavaScript object declarations

- Keys
- Values

```
{ "name": "Barry", "age": 42, "email": "barry@plymouth.ac.uk" }
```

Convert the object to a string so that it can be stored in a file or transmitted across the internet

Data representation (client-side)

HTML can be added to the DOM directly

```
let elem = document.getElementById("document");
elem.innerHTML = response.responseText;
```

JSON structured by the JSON.parse method – converted to a JavaScript object

```
let data = JSON.parse(response.responseText);
```

XML data can be queried like the DOM (used less frequently)

```
let x = xmlhttp.responseXML.getElementsByTagName("PERSON");
```

Back-end return JSON

WARNING: does not work yet - CORS

Front-end (client-side)

index.html

```html
<html>
  <head>
    <title>Random Number</title>
    <script src="main.js"></script>
  </head>
  <body>
    <p>Customers:</p>
    <div id="divCust">loading...</div>
  </body>
</html>
```

main.js

```javascript
window.onload = function() {
  let request = new XMLHttpRequest();
  request.onreadystatechange = function() {
    if (this.readyState === 4 && this.status == 200) {
      let elem = document.getElementById("divCust");
      let customers = JSON.parse(this.responseText);
      elem.innerHTML = customers[0].surname;
    }
  };
  request.open("GET", "http://localhost:82/", true);
  request.send();
}
```

JSON to object

Back-end (server-side)

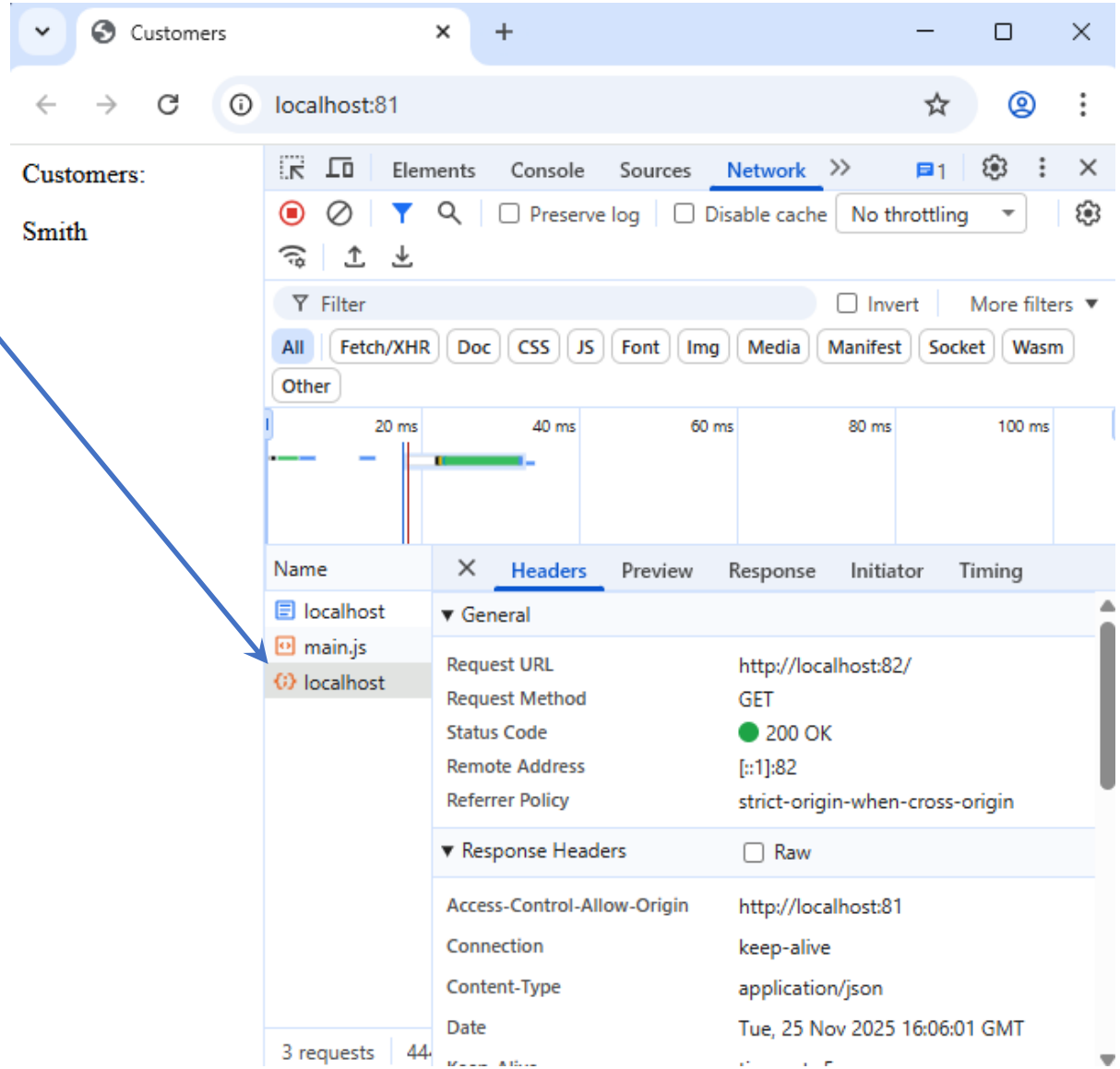
server.js

```javascript
let http = require("http");
let port = 9000;

let server = http.createServer(
  function(request, response) {
    var customers = [
        {"surname": "Smith"},
        {"surname": "Jones"}];
    response.writeHead(200,
        {'Content-Type': 'application/json'});
    response.end(JSON.stringify(customers));
  }
);

server.listen(port, function() {
  console.log("Server listening on port " + port);
});
```

object to JSON

Debugging Network Traffic between Containers

- F12
  (Inspect)
- Can see Network traffic:



Exercise 1: Connecting Docker Containers using Ajax

- Use the Ajax code (provided earlier in these slides) to connect two applications running in separate containers (1 Nginx container and 1 Node.js container). This essentially joins the material from the previous two session's exercises.
- Note: you need to add a header to the backend (Node.js) code, to allow the front-end to connect to it (as shown in this session's slides).
- Alter your web-service API (Node.js application) to return JSON from an endpoint, and use this in your other (NginX) application / container

Exercise 2: Ajax Trains

- In this exercise you will construct an application that uses Ajax to retrieve and display information from an existing webservice API – type the following URL into a browser (such as Google Chrome):
  https://web.socem.plymouth.ac.uk/COMP3006/trains/trains
  (the data describes the trains that will call at a station)

- Your job is to create a display board (in a web browser) for use at the train station.

- The first step is to construct the basic web page you will use. Create a new file called ***index.html*** and:
  - Give the page a title (e.g., “Station Sign”).
  - Add script and style elements for your own code.
  - Add a body element to the page.

- Once you have set up the page, modify the body element as follows:

```
<body>
    <table>
        <thead><tr>
            <th>Time</th><th>Destination</th><th>Platform</th>
        </tr></thead>
        <tbody id="tabTrains"></tbody>
    </table>
</body>
```

you should create a .js & .css file and link them to the web page

Exercise 2: Ajax Trains

- The tbody ID attribute is particularly important (it identifies the HTML element which receives the data retrieved from the web-service API)

- Having set up the HTML, you can construct the JavaScript code that will retrieve and display the data. Within the script element (or in a linked .js file) add the following code (which should display the table on the right):

```
window.onload = function() {
    let request = new XMLHttpRequest();
    request.onreadystatechange = function() {
      if (this.readyState === 4 && this.status == 200) {
        let elem = document.getElementById("tabTrains");
        let trains = JSON.parse(this.responseText);
        let rows = "";
        for (i=0; i<trains.trains.length; i++) {
            let train = trains.trains[i];
            rows += "<tr>"
                + "<td>" + train.departs + "</td>"
                + "</tr>";
        }
        elem.innerHTML = rows;
      }
    };

    let url = "https://web.socem.plymouth.ac.uk/COMP3006/trains/trains";
    request.open("GET", url, true);
    request.send();
  }
```

Time Destination Platform
0947
0953
1003
1004
1007
1022
1032
1034
1039
1041
1051
1101
1104
1105
1132
1137
1140
1150

- Add code to display the platform.

- Add code to display the destination (which will be the last stop).

Exercise 2: Ajax Trains

- The final part of this exercise is to add some CSS to style the page. You should implement the following styling:
  - The text should be 32px in the Arial font and should be left-aligned.
  - The header of the table should have a blue background with white text.
  - The body of the table should have a black background. The time and platform columns should have white text, and the destination column (identified by the class `destination`) should have yellow text.
  - The table should take up the whole width of the display. The border collapse property should be set to `collapse` to remove spaces between the cells.
  - The time and platform columns should have a width of 20% of the display, while the destination should take up 60%.
  - Every cell in the table should have 5px of padding.

the CSS rules to complete this exercise have been seen elsewhere in the module – look back at lectures and previous workshops

Exercise 3: Objects

- This week is all about objects, so the first thing to do is to make sure you can define your own objects. Write an object that stores your forename, surname and date of birth in an object. Use the **console.log** pattern to print out the whole object, and each key/value pair.
- Define the following data structure (you can use constructors or classes). A lecturer has a name, email address and office number. A student has a name, email address and student number.
- Examples are:
  - Lecturer: Mark Dixon, mark.dixon@plymouth.ac.uk, PSQ A319
  - Student: Jane Smith, jane.smith@students.plymouth.ac.uk, 1234567
- Your task is to extract the details from a form, use them to instantiate an object of either type Lecturer or Student, and then use the object to populate a table.
  - You should abstract the common data to a class or object called Person, which is inherited by the Lecturer or Student.
  - In the HTML you should have a form for adding a student and a form for adding a lecturer. There should be a table for each type too (you will need to give the table an ID so that you can add rows to it).
  - Each form should have an add button so that the current form values are used to create an object of the appropriate type, before being added to the appropriate table when the button is clicked. The last operation in the event handler should clear the form for the next new record.
- Once you have it working you should extend the code so that new items appear in a page using HTML.